

PHENIKAA UNIVERSITY

PHENIKAA SCHOOL OF COMPUTING



COURSE: SOFTWARE ARCHITECTURE

TOPIC:

QR CODE-BASED IN-TABLE FOOD ORDERING SYSTEM REPORT

LAB 2 – LAYERED ARCHITECTURE DESIGN

DOCUMENT: SOFTWARE ARCHITECTURE

Code Course : CSE703110-1-3-25

Class : N01

Instructor : THS. Vũ Quang Dũng

Group : 01

Members : 1. Nguyễn Đình Trung - 23010701
2. Vũ Anh Nam - 23010886
3. Đặng Minh Hoàng – 23010445
4. Nguyễn Phan Quang Minh - 23010530

Table of Contents

1. Abstract.....	3
2. Layered Architecture Principles.....	3
3. Definition of the Four Layers and Their Responsibilities in the Training Program Management System.....	3
4. Demonstration Scenario: View Training Program (UC-3.27)	4
5. UML Component Diagram.....	8
6. Conclusion.....	10

1. Abstract

Description of Lab 02

This lab explores the application of the Layered Architecture model to organize the system into structured and maintainable layers. The objective is to analyze how these layers interact when a specific feature is executed and to represent the architectural design through a UML Component Diagram.

For demonstration, the feature “**Browse & View Menu Details**” (UC2) is selected from our project. This feature effectively illustrates the separation of responsibilities, data flow across layers, and the logical relationships among architectural components within the system.

2. Layered Architecture Principles

The system is structured into four layers: **Presentation** → **Business Logic** → **Persistence** → **Data**. A fundamental rule of the architecture is that each layer may only communicate with the layer directly beneath it, avoiding any shortcut calls to deeper layers. This hierarchical organization enhances maintainability, supports scalability, and minimizes cross-layer dependencies, ensuring that changes in one layer have limited impact on others.

3. Definition of the Four Layers and Their Responsibilities in the QR Code Food Ordering System

a. Presentation Layer

This layer is responsible for receiving user requests (such as scanning table QR codes or clicking to view menu items), directing the processing flow, and returning JSON responses or web views to the client application. It acts as the entry point to the system and interacts strictly with the Business Logic Layer.

Representative :

- MenuController

b. Business Logic Layer

This layer handles all domain-specific operations related to the food ordering system, such as calculating active menu items, cross-checking real-time availability, applying discount rules, and structuring the data before response mapping. It defines and

enforces the system's business rules.

Representative :

- MenuService

c. Persistence Layer

This layer is responsible for querying and retrieving raw data from the database based on the criteria specified by the Business Logic Layer. It abstracts data access using the Repository pattern and ensures that business logic remains independent from underlying database details.

Representative :

- MenuRepository

d. Data Layer

The Data Layer contains the physical database schemas and tables that persist all information related to menus, food items, categories, custom order notes, and transaction statuses. It is accessed only through the Persistence Layer..

Example tables include:

- menu, food_category, food_item, table_session, order_bill

4. Demonstration Scenario: View Menu Details (UC2)

The execution of **UC2: Browse & View Menu Details** follows the standard Layered Architecture flow, in which each layer interacts only with the layer directly below it. The request originates from the customer's web interface (triggered by a QR code scan or manual navigation) and moves downward through the layers to fetch menu entities, before propagating back upward with the compiled data.

Standard flow:

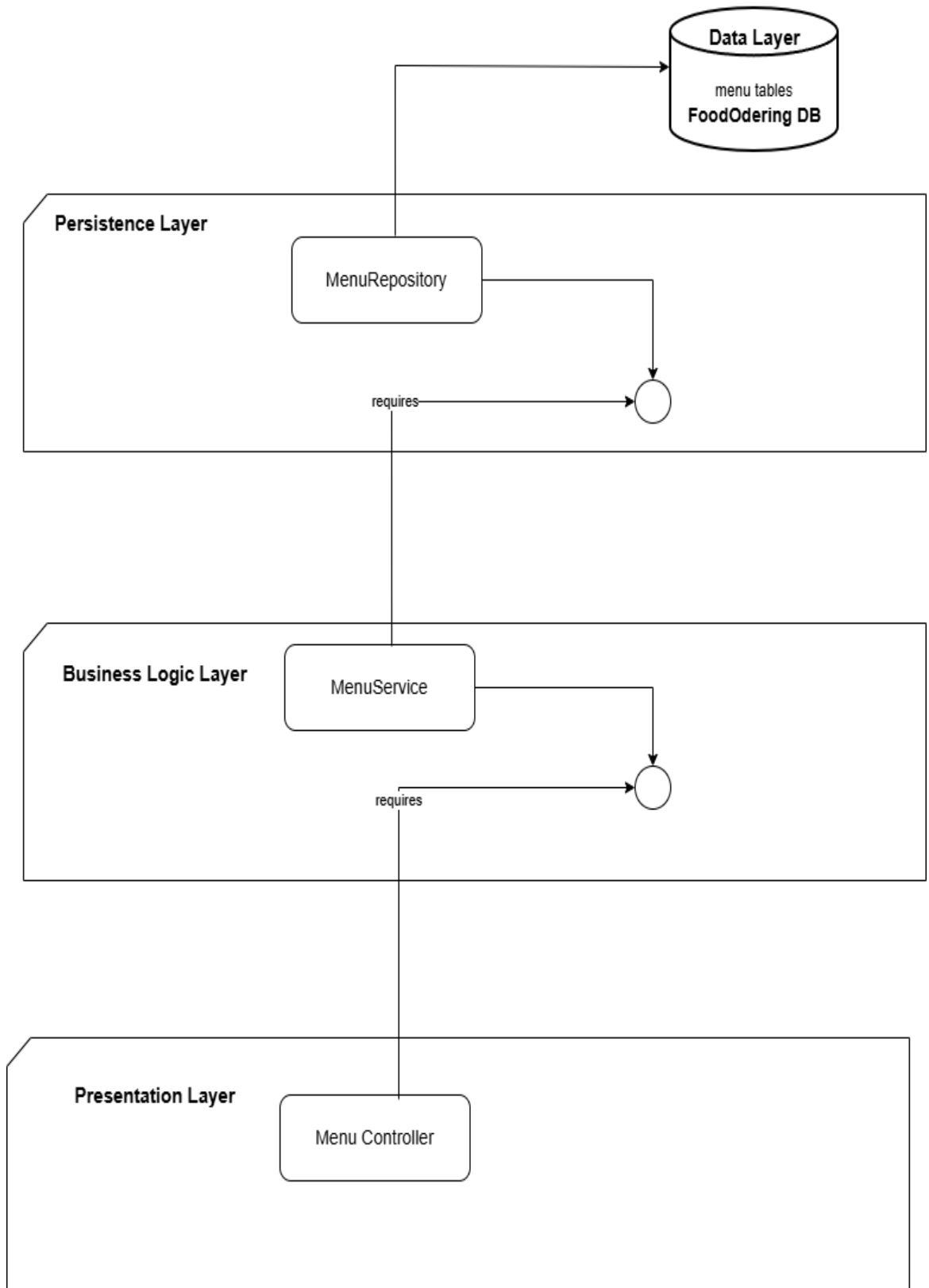
Client → L1 (Presentation) → L2 (Business Logic) → L3 (Persistence) → L4 (Data)
→ L3 → L2 → L1 → Client

Applied to UC2:

- A Web Customer initiates the request by scanning a table QR code or navigating the site, which is received by the MenuController (L1).
- The controller validates input parameters (e.g., menu_id, table_number) and forwards the request to the MenuService (L2).
- The service applies business rules (e.g., verifying if the restaurant menu is currently active) and queries the MenuRepository (L3).
- The repository executes data mappings from the physical database schema (L4).
- The result propagates back along the reverse path: Repository → Service → Controller, compiling the data safely into a DTO before rendering it on the user's mobile screen

5. UML Component Diagram

Lab 2: Layered Architecture - Component Diagram



The component diagram is organized into three principal blocks corresponding to the architecture layers:

- **Presentation**
 - Component: MenuController
 - Required Interface: IMenuService
- **Business Logic**
 - Component: MenuService
 - Provided Interface: IMenuService
 - Required Interface: IMenuRepository
- **Persistence**
 - Component: MenuRepository
 - Provided Interface: IMenuRepository
 - Connection: Database (FoodOrderingDB)

The Component Diagram models the logical structure of the system according to the Layered Architecture, explicitly showing dependency flow from Presentation → Business Logic → Persistence → Data. For the illustrative use case **UC2: Browse & View Menu Details**, the primary components are:

- **MenuController (Presentation Layer):** Receives user requests (e.g., GET /api/menu/{id} triggered by a QR scan), validates and normalizes input parameters (such as menuId and tableNumber), and invokes the business contract exposed via IMenuService.
- **MenuService (Business Logic Layer):** Implements IMenuService. It orchestrates domain logic for assembling menu details: fetching base menu data, associated food items, categories, and real-time availability information. It delegates raw data retrieval to IMenuRepository and maps persistence entities into the MenuDetailDTO returned to the Presentation Layer.
- **MenuRepository (Persistence Layer):** Implements IMenuRepository. It encapsulates all data access, executing queries against the physical database (FoodOrderingDB) and returning persistence entities or projections used by the service.

Using interface contracts (IMenuService and IMenuRepository) decouples components, reduces tight coupling, and enforces the layered constraint that each layer may only interact with the layer directly beneath it. This design improves testability (components can be tested via mocks of their interfaces), maintainability, and future replaceability of persistence technologies.

6. Conclusion

Through the implementation of the "Browse & View Menu Details" feature (UC2), the team successfully applied the Layered Architecture model to demonstrate how a request is processed from the Presentation Layer down to the Data Layer. The clear separation of responsibilities across layers ensures strict dependency control, improves maintainability, and supports consistent system behavior.

The architectural structure established in this lab forms a strong foundation for the next phase, Lab 3: Layered Architecture Implementation (CRUD). The defined components—MenuController, MenuService, and MenuRepository—along with their interface contracts, will be directly implemented in Lab 3 to handle real CRUD operations. This continuity ensures that code-level implementations follow the Logical View defined here, preserving modularity, testability, and extensibility. With this layered model in place, additional features such as shopping cart management and payment processing can be implemented efficiently in subsequent iterations.

